Pemrograman Web Lanjut

SIB245007 | D-IV Sistem Informasi Bisnis

Pertemuan 9: **Pengolahan Data: Impor, Ekspor, dan Antrean**

Dari rekap laporan, berkas CSV, sampai pekerjaan berat yang dititipkan ke latar belakang

Yang Akan Kamu Pelajari

1. Menyusun laporan yang merekap data dalam rentang tanggal, dan menghindari jebakan umum saat memfilter tanggal
2. Mengekspor data ke CSV tanpa membebani memori server, dan mengimpor CSV dengan menangani baris yang cacat tanpa menghentikan seluruh proses
3. Mengenali kapan sebuah pekerjaan sebaiknya dititipkan ke **antrean (queue)** alih-alih dikerjakan langsung selama pengguna menunggu

> Slide ini membahas konsep. Menerapkan laporan, ekspor/impor, dan antrean pada Simple POS dikerjakan di jobsheet praktikum.

Peta Pertemuan: Seperti Dapur Restoran

Data mentah → **Laporan** → **CSV keluar / masuk** → **Antrean**

> Bayangkan dapur restoran. Bahan mentah masuk (**extract**), dimasak jadi hidangan (**transform**), lalu disajikan ke meja (**load**). Kalau pesanan membludak, dapur tidak menyuruh tamu berdiri di depan kompor: tamu diberi nomor meja, lalu hidangan diantar belakangan. Itulah gambaran besar pertemuan ini.

Bagian 1

Laporan dan Agregasi Data

Empat Istilah Dasar

ETL (Extract, Transform, Load): pola umum mengambil data mentah, mengubah bentuknya, lalu menyimpan atau menyajikannya dalam bentuk baru.

Batch Processing: memproses banyak data sekaligus dalam kelompok, bukan satu per satu.

Antrean (Queue): mekanisme menitipkan pekerjaan berat agar dikerjakan di latar belakang oleh proses terpisah, bukan langsung selama request pengguna berlangsung.

Import/Export: membawa data masuk dari format eksternal (CSV, Excel) ke basis data, atau mengeluarkannya ke format eksternal untuk dibaca di luar aplikasi.

Daftar vs Laporan: Tumpukan Struk vs Rekap Akhir Bulan

Halaman daftar itu seperti tumpukan struk di laci kasir: semua ada, tapi kamu yang harus menghitungnya sendiri. Halaman laporan itu seperti rekap akhir bulan: sudah dijumlahkan dan siap dibaca. Pertanyaannya lebih sempit tapi lebih berat: "berapa total X pada rentang waktu tertentu".

```
$totalKata = Article::whereBetween('published_at', [
    $from, $to,
])->sum('word_count');
```

`whereBetween` memfilter baris yang tanggalnya berada di antara dua batas, lalu `sum()` menjumlahkan satu kolom dari hasil filter itu. Penjumlahan dikerjakan oleh database, jadi Laravel tidak perlu memuat ribuan baris ke memori hanya untuk dijumlahkan.

Jebakan Umum: Batas Hari yang Terpotong

> Filter "1 Agustus sampai 31 Agustus" yang ditulis apa adanya berarti "sampai jam 00:00:00 tanggal 31". Seluruh transaksi yang terjadi sepanjang tanggal 31 ikut hilang, seperti toko yang menghitung omzet bulan Agustus tapi berhenti saat baru buka pintu di hari terakhir.

```
$from = $request->date('from')->startOfDay();
$to = $request->date('to')->endOfDay();
```

`startOfDay()` mengubah tanggal awal menjadi jam 00:00:00, dan `endOfDay()` mengubah tanggal akhir menjadi jam 23:59:59, supaya seluruh hari terakhir ikut terhitung.

Jebakan Batas Hari, Digambar di Garis Waktu



Total yang Sudah Tersimpan, Tidak Dihitung Ulang

Pertemuan 6 menegaskan: nilai seperti total transaksi tidak boleh dipercaya kalau datang dari input klien, dan harus dihitung ulang di server saat transaksi dibuat. Begitu nilai itu tersimpan dengan benar, laporan boleh memakainya apa adanya. Ini seperti total di nota yang sudah dicetak: saat rekap, kamu cukup menjumlahkan angka di nota, bukan mengecek ulang tiap item di dalamnya.

```
// Laporan cukup menjumlahkan kolom yang sudah tersimpan
$totalPenjualan = Transaction::whereBetween('created_at', [$from, $to])
    ->sum('total');
```

> Menghitung ulang total dari baris detail satu per satu, setiap kali laporan dibuka, membebani server tanpa alasan: angkanya sudah benar sejak transaksi itu dibuat.

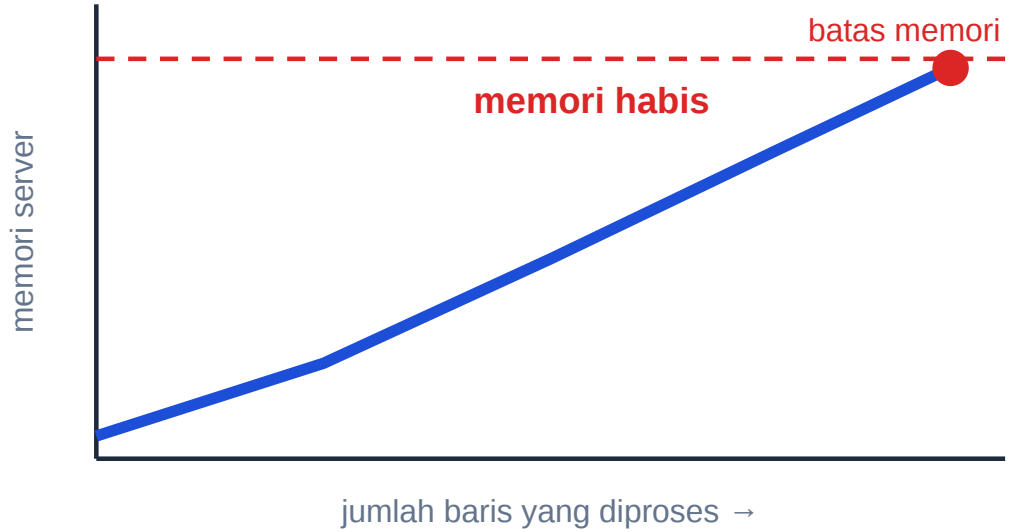
Bagian 2

Ekspor dan Impor CSV

Ember vs Keran: Dua Cara Mengirim Berkas Besar

Ember: tampung dulu, kirim belakangan

get() lalu susun satu string besar



Keran: alirkan baris demi baris

lazyById() + streamDownload()



Ekspor dengan `streamDownload()` dan `lazyById()`

Cara keran: ambil data per potongan kecil, tulis ke output, lalu lupakan.

```php
$articles = Article::with('author')
    ->whereBetween('published_at', [$from, $to]);

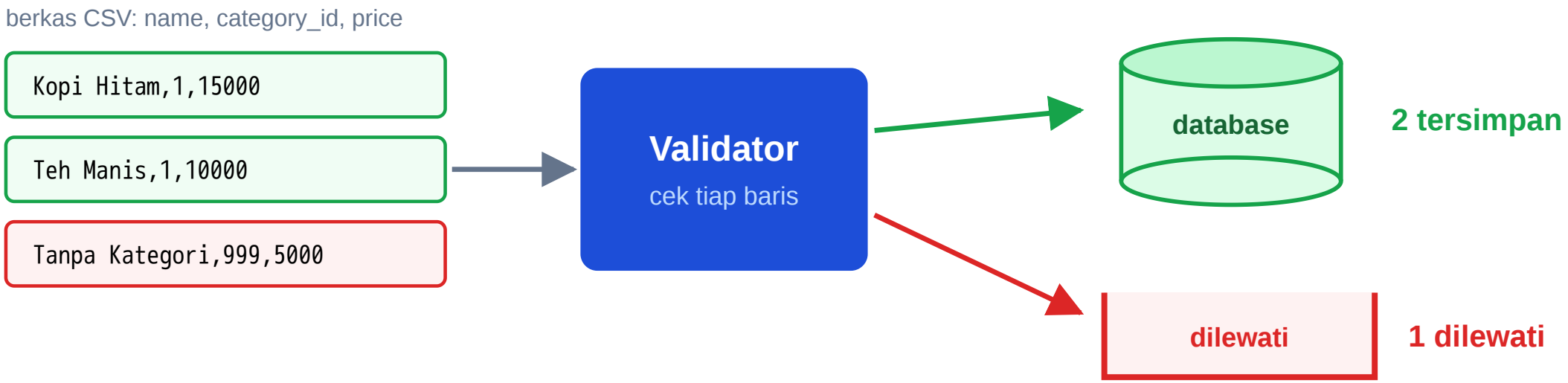
return response()->streamDownload(function () use ($articles) {
    $out = fopen('php://output', 'w');
    fputcsv($out, ['Judul', 'Penulis', 'Tanggal']);
    foreach ($articles->lazyById() as $a) {
        fputcsv($out, [$a->title, $a->author->name, $a->published_at]);
    }
    fclose($out);
}, 'laporan-artikel.csv');
```

`lazyById()` membaca database per potongan, dan `streamDownload()` mengirim tiap baris sambil jalan, jadi memori tetap datar.

> Laravel docs: `HTTP Responses → File Downloads` dan `Eloquent → Chunking / lazyById()`

Impor: Seperti Kasir yang Memindai Belanjaan

Impor berjalan ke arah sebaliknya: membaca berkas CSV, lalu mengubah tiap barisnya jadi baris baru di database. Bayangkan kasir memindai belanjaan. Kalau satu barcode rusak, kasir menyisihkan barang itu dan terus memindai sisanya, bukan menyuruh semua pelanggan pulang.



satu baris cacat tidak menghentikan baris lain

Impor: Validasi Tiap Baris, Lewati yang Cacat

```php
$rows = file($path, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
$rows = array_map('str_getcsv', $rows);
$header = array_shift($rows);

foreach ($rows as $row) {
    $validator = Validator::make(array_combine($header, $row), [
        'title' => ['required', 'string'],
        'author_id' => ['required', 'exists:authors,id'],
    ]);

    if ($validator->fails()) {
        continue; // lewati baris ini, lanjut ke baris berikutnya
    }

    Article::create($validator->validated());
}
```

CSV dari Excel versi Indonesia sering memakai titik koma (`;`) sebagai pemisah dan menyisipkan karakter tak terlihat (BOM) di awal berkas. Akibatnya header pertama tidak terbaca dan semua baris gagal validasi. Cek header-nya dulu kalau semua baris tiba-tiba ditolak.

Bagian 3

Kapan Memakai Antrean

Dua Cara Memproses Pekerjaan

Langsung (dalam request)

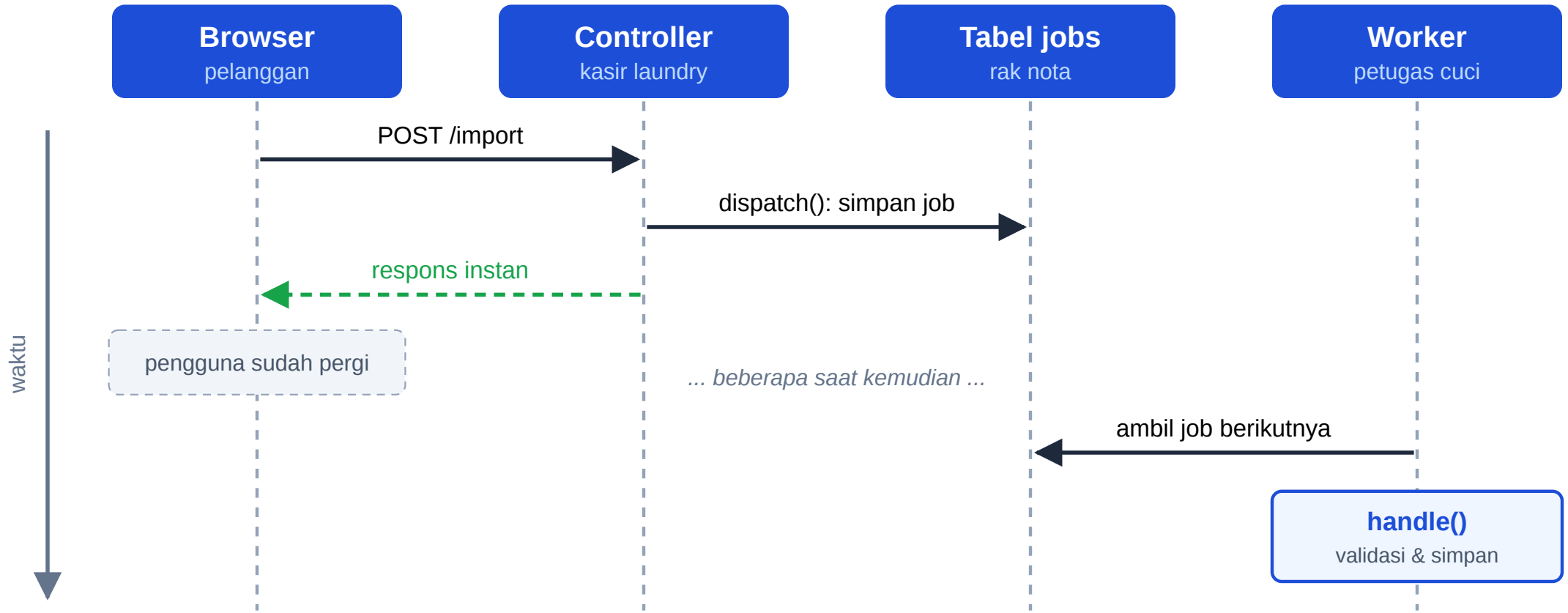
- Pengguna menunggu sampai selesai
- Kalau gagal, pengguna langsung tahu
- Cocok untuk pekerjaan singkat, hasilnya dibutuhkan seketika

Lewat Antrean (job)

- Respons instan, pekerjaan berjalan di latar belakang
- Pengguna tidak menunggu di depan browser
- Cocok untuk pekerjaan berat atau berdurasi tidak pasti

> Analogi laundry: ada cucian satu kaus, kamu rela menunggu di tempat. Ada cucian satu karung, kamu titip, dapat nota, lalu pulang. Impor puluhan baris itu satu kaus. Impor ribuan baris itu satu karung, dan permintaan HTTP punya batas waktu tunggu.

Antrean = Laundry Kiloan



Alur Job di Laravel

Controller → **dispatch() job** → **Tabel jobs** → **Worker (queue:work)**

```
php artisan make:job ProcessArticleImport
```

```
ProcessArticleImport::dispatch($csvPath);

return back()->with('success', 'Impor sedang diproses di latar belakang.');
```

Controller hanya menyimpan berkas dan mengirim satu job ke antrean, lalu segera merespons pengguna. Pekerjaan sesungguhnya (loop validasi dan penyimpanan) pindah ke metode `handle()` pada class job itu.

Worker: Petugas yang Mengerjakan Tumpukan Nota

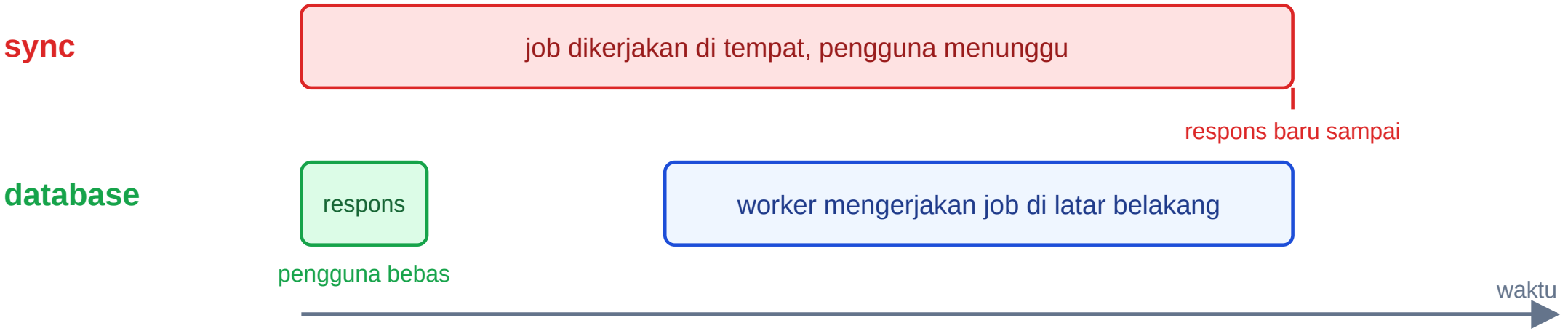
> **Worker:** proses PHP terpisah yang terus berjalan, mengambil job dari tabel `jobs` satu per satu, lalu menjalankan metode `handle()` -nya.

```
php artisan queue:work
```

Tanpa worker, job hanya menumpuk di tabel `jobs`. Sama seperti laundry tanpa petugas cuci: nota terus bertambah, tapi tidak ada satu pun cucian yang selesai, karena `dispatch()` hanya menyimpan nota, bukan mencuci.

> Kalau `QUEUE_CONNECTION` di `.env` bertuliskan `sync`, job dikerjakan langsung di tempat dan tidak pernah masuk antrean sungguhan. Itu seperti kasir yang menyuruhmu menunggu sampai cucian kering padahal kamu sudah memegang nota.

`sync` vs `database`: Siapa yang Menunggu?



Tanda-Tanda Pekerjaan Layak Masuk Antrean

- Durasinya tidak bisa diprediksi di muka, tergantung ukuran data yang diproses
- Hasilnya tidak perlu dilihat pengguna seketika di halaman yang sama
- Melibatkan panggilan ke layanan eksternal yang lambat, misalnya mengirim email atau memanggil API pihak ketiga

> Laporan yang dibuka dan dibaca langsung di halaman biasanya tetap lebih baik dikerjakan langsung, karena pengguna memang menunggu hasilnya saat itu juga. Antrean bukan solusi untuk semua pekerjaan berat, hanya untuk yang tidak butuh hasil instan.

Kalau Job Gagal: Catatan Komplain Laundry

Cucian yang bermasalah tidak dibuang diam-diam. Job yang gagal dicatat di tabel `failed_jobs` lengkap dengan pesan errornya, dan bisa dicoba ulang setelah masalahnya diperbaiki.

```
php artisan queue:failed        # lihat daftar job yang gagal
php artisan queue:retry all     # coba jalankan ulang semuanya
```

```php
class ProcessArticleImport implements ShouldQueue
{
    use Queueable;

    public int $tries = 3; // maksimal 3 kali percobaan
}
```

> Kalau job "sudah jalan tapi hasilnya kosong", cek `queue:failed` lebih dulu: pesan errornya hampir selalu ada di sana.

Laravel docs: `Queues → Dealing With Failed Jobs`

Menerapkan pada Simple POS

Konsep laporan, ekspor/impor CSV, dan antrean ini akan kamu terapkan langsung pada Simple POS di jobsheet praktikum: menyusun laporan penjualan dengan filter tanggal, mengekspor laporan itu ke CSV, membangun impor produk dari CSV, lalu memindahkan impor itu ke job antrean supaya pengguna tidak menunggu di depan browser.

> Kode lengkap: `github.com/se-polinema/simple-pos-ch08`

Rangkuman (1/2)

- Laporan merekap data dalam rentang tanggal lewat `whereBetween`, dan agregasinya (`sum()`) sebaiknya dikerjakan oleh database
- `startOfDay()` / `endOfDay()` mencegah hari terakhir terpotong di tengah malam
- Nilai yang sudah tersimpan dengan benar (seperti total transaksi) boleh dipakai apa adanya, tidak perlu dihitung ulang setiap laporan dibuka
- Ekspor CSV memakai `lazyById()` dan `streamDownload()`: seperti keran, bukan ember, sehingga memori tetap datar

Rangkuman (2/2)

- Impor CSV memvalidasi tiap baris dan melewati yang gagal lewat `continue`, bukan menghentikan seluruh proses
- Pekerjaan berdurasi tidak terprediksi atau bervolume besar sebaiknya dititipkan ke job antrean, seperti laundry kiloan: respons instan, pekerjaan menyusul
- Job yang di-dispatch hanya masuk tabel `jobs`, dan baru berjalan setelah worker (`queue:work`) memprosesnya
- Job yang gagal tercatat di `failed_jobs` dan bisa dicoba ulang dengan `queue:retry`

Referensi & Diskusi

Dokumentasi resmi Laravel (Queues, File Storage, HTTP Responses)

Kode lengkap: `github.com/se-polinema/simple-pos-ch08`

Pertemuan berikutnya: Merancang dan Membangun REST API